

Bancos de Dados Relacionais e Não Relacionais



Conteudista: Prof. Esp. Jônatas Carvalho

Revisão Textual: Esp. Anna Carolina Guimarães

Objetivo da Unidade:

- Compreender os conceitos de bancos de dados, tanto relacionais quanto não relacionais e suas integrações com o Node.js.

 **Material Teórico**

 **Referências**



Material Teórico

Introdução

Os bancos de dados são peças essenciais em um sistema, pois por meio deles podemos armazenar dados e guardar informações valiosas para o produto em que trabalhamos. Existem diversos tipos de banco de dados, porém, no mercado, a maioria dos sistemas utilizam dois tipos, que são: banco de dados relacionais e banco dados não relacionais.

Um banco de dados relacional trabalha com tabelas relacionais, ou seja, tabelas que serão compostas por linhas e colunas. Essas tabelas funcionam de forma parecida com as planilhas do Excel, sendo que cada coluna irá representar o atributo ou a característica do registro, e a linha representa o registro individual de uma entidade.

A capacidade de estabelecer relacionamentos entre tabelas é a principal característica de um banco de dados relacional. Esses relacionamentos acontecem através das chaves primárias e estrangeiras que são estabelecidas no momento da criação das bases ou após alguma atualização nessas bases. Esses relacionamentos entre tabelas garantem que as consultas aos dados sejam mais eficientes e mantenham a integridade dos dados.

Ocasões em que necessitamos ter organização e consistência, são o cenário ideal para a utilização de bancos de dados relacionais, ou seja, quando os dados têm estrutura definida e entidades que podem se relacionar de forma clara. Dessa forma, são indicados para situações em que é crítica a integridade dos dados e para qualquer tipo de consulta complexa, como junções, agrupamentos e cálculos. Por fim, também

possuem recursos de conformidade e segurança avançados, podendo ser eficazes para atender padrões rigorosos de normas e regulamentos.

Os bancos relacionais normalmente são utilizados nos SGBDs ou Sistema de Gerenciamento de Banco de Dados juntos com a linguagem SQL ou *Structured Query Language* (Linguagem de Consulta Estruturada), com isso, normalmente veremos esses termos sempre utilizados juntos, porém são coisas diferentes.

Agora, falando sobre bancos de dados não relacionais, vamos muito encontrar o termo NoSQL, que diferentemente de SQL, não é uma linguagem. NoSQL é um termo que é utilizado para referenciar bancos de dados não relacionais. Esse modelo de banco de dados não relacional pode implementar uma variedade de modelos, que são eles:

- **Grafos:** representação de dados interconectadas para possíveis análises detalhadas de relações. Um banco de dados que utiliza esse modelo é o Neo4j, muito utilizado no site Medium. O modelo de grafos poderia ser muito útil em um sistema de recomendações nas redes sociais, podendo analisar as conexões entre amigos e conteúdos podendo fazer sugestões, por exemplo, de novos amigos, novos conteúdos e até mesmo de promoções;
- **Orientado a documentos:** o armazenamento dos dados é realizado em documentos no formato json, na qual, cada documento possui uma chave única para identificação. É muito utilizado em aplicações que necessitam de uma flexibilidade na estrutura de dados e que utilizam um grande armazenamento de informações. O MongoDB implementa esse tipo de modelo e pode ser muito útil para ser utilizado em um sistema de uma loja on-line na qual pode armazenar detalhes personalizados para cada produto;
- **Chave valor:** cada dado armazenado é identificado por um par de chave e valor, na qual cada chave é um id único e o valor são os dados que se deseja armazenar. O banco de dados Redis é muito utilizado nesse modelo que é implementado normalmente para caches, na qual precisamos de uma leitura e uma gravação de dados muito rápida. Um exemplo de implementação utilizando esse modelo pode ser um sistema que realiza o armazenamento das informações de um usuário

deixando assim mais rápido o carregamento de páginas que previamente ele pode ter acessado;

- **Modelo colunar:** armazena os dados por colunas, isso faz com que as consultas de grandes volumes de dados sejam otimizadas. Esse modelo é utilizado pela Netflix utilizando o banco de dados Cassandra. Um sistema que podemos dar de exemplo seria uma plataforma que análise de logs gerados por outros sistemas, podendo realizar buscas apenas pelas colunas de *timestamp* e erro para poder identificar falhas rapidamente.

Pelos exemplos dados anteriormente, percebemos que bancos de dados não relacionais nos dão soluções flexíveis e otimizadas, na qual podem atender diversos cenários, podendo estar mais focado em performance, organização dos dados ou escalabilidade.

Vale lembrar que bancos de dados relacionais e não relacionais não são rivais, e em aplicações e sistemas de grande porte sempre são utilizados para se complementar a fim de resolver as complexidades que os sistemas atuais implementam.

Criação e Principais Características da Linguagem SQL

Em 1970, Edgar F. Codd publicou um artigo chamado “*A Relational Model of Data for Large Shared Data Banks*”, em português “Um Modelo Relacional de Dados para Grandes Bancos de Dados Compartilhados” que introduziu o conceito de banco de dados que realizam a organização dos dados através de tabelas composta por linhas e colunas que são ligadas por relacionamentos. Na época foi muito revolucionária essa ideia, pois era uma alternativa aos modelos hierárquicos ou em rede que eram os modelos de armazenamento da época que eram mais complicados e menos flexíveis. A fim de validar os conceitos de Codd na vida real, a IBM criou o projeto System R no seu laboratório de pesquisa em San Jose, Califórnia. Sua intenção era projetar um protótipo de sistema de gestão de banco de dados relacional que aconteça ser tanto funcional como eficaz. Dentro deste projeto, foi criada uma linguagem conhecida

como SEQUEL, abreviação de *Query Language* em português Linguagem de Consulta, que posteriormente virou SQL.

Structured Query Language ou SQL, em português “Linguagem de Consulta Estruturada” é uma linguagem padronizada utilizada para realizar o gerenciamento, manipulação e consultas de dados que estão armazenados em sistemas de banco de dados relacionais. O SQL foi criado para ser de fácil aprendizado e fácil de usar, permitindo que usuários com uma curva de aprendizado pequena consiga realizar operações como consultas, atualizações, inserções e exclusões de dados.

A linguagem é padronizada pelo ANSI que é o Instituto Nacional de Padronização dos Estados Unidos e pela ISO que é a Organização Internacional de Padronização, por ser padronizada por essas duas instituições o SQL é muito utilizada em diversos Sistemas de Gerenciamento de Banco de Dados (SGBDs), como MySQL, PostgreSQL, MariaDB e Oracle. Todos esses SGBDs implementam o SQL, porém cada uma possui algumas especificações e singularidades, ambas são muito parecidas e a curva de aprendizado quando se usa um SDBG e se muda para outro não há grandes dificuldades.

Tipos de Instruções

Segundo Shannon Bradshaw e Eoin Brazil (2019), a linguagem SQL possui instruções que podem ser divididas em diferentes categorias que estão relacionadas com o objetivo de cada grupo de instrução.

O primeiro tipo de instruções se chama *DDL* ou *Data Definition Language*, em português “Linguagem de Definição de Dados”. Essas instruções são as responsáveis por cuidar da definição e organização da estrutura do banco de dados. Através das instruções classificadas como DDL, definiremos como o banco de

dados será estruturado e podemos realizar a criação de tabelas, modificá-las e também as excluir. Os principais comandos DDL são: *CREATE*, *ALTER*, *DROP*, *RENAME*.

O segundo tipo de instruções se chama *DML* ou *Data Manipulation Language*, em português “Linguagem de Manipulação de Dados”. Essas instruções nos permitem realizar inserção de novos dados, atualização dos dados e remoção dos dados. Os principais comandos DML são: *SELECT*, *INSERT*, *UPDATE*, *DELETE*, *MERGE*.

O terceiro tipo de instruções se chama *DCL* ou *Data Control Language*, em português “Linguagem de Controle de Dados”. Essas instruções são voltadas para o controle de acesso na qual permite definir quais pessoas podem realizar o acesso e a manipulação dos dados. Esses comandos são mais utilizados por equipes de infra e são fundamentais para garantir a segurança dos dados, pois através deles permitimos e revogamos os acessos, assim evitando fraudes e vazamento de informações delicadas que estão armazenadas. Os principais comandos DCL são: *GRANT*, *REVOKE*.

O quarto tipo de instruções se chama *TCL* ou *Transaction Control Language* em português “Linguagem de Controle de Transação”. Essas instruções têm como objetivo realizar o gerenciamento das transações do banco de dados, são elas as responsáveis pela consistência dos dados, ou seja, garante que os dados estejam

corretos, precisos e atualizados. Os principais comandos TCL são: *COMMIT*, *ROLLBACK*, *SAVEPOINT*.

Cada uma dessas categorias possuem um papel essencial para garantir que o SQL não pareça apenas uma linguagem de consulta, mas sim uma ferramenta concisa e completa quando o assunto é gerenciamento de banco de dados. Esses grupos de instruções acima representados devem trabalhar juntos para criação, manipulação, controle e proteção de dados de forma eficiente.

Integração Entre uma Aplicação em Node.js e o MySQL

Faremos agora uma integração do Node.js com um banco de dados MySQL. Vamos utilizar como base nosso conversor de números decimais para binário na qual construímos em nossa primeira unidade, porém vamos fazer algumas refatorações. Novamente, crie um diretório em seu computador e nesse diretório crie um arquivo chamado `index.js` e insira o código abaixo:

```
const express = require("express");
const mysql = require("mysql2");
const app = express();
const port = 3000;

// Configuração da conexão com o banco de dados MySQL
// no contêiner Docker

const db = mysql.createConnection({
  host: "localhost", // Use 'mysql-container' se o Node.js
```

```

também estiver em um contêiner
    user: "user", // Definido no docker-compose.yml
    password: "userpassword", // Definido no docker-compose.yml
    database: "conversoes_db",
  });

// Conectar ao banco de dados

db.connect((err) => {
  if (err) {
    console.error("Erro ao conectar ao banco de dados:", err);
    return;
  }
  console.log("Conectado ao banco de dados MySQL (Docker).");
});

// Endpoint para converter decimal para binário e salvar no banco de dados
app.get("/to-binary/:decimal", (req, res) => {
  const decimal = parseInt(req.params.decimal, 10);
  if (isNaN(decimal)) {
    return res.status(400).json(
      ({ error: "Número decimal inválido" });
    );

    const binary = decimal.toString(2);
    // Salvar no banco de dados
    const query = "INSERT INTO conversoes
(numero_decimal, numero_binario) VALUES (?, ?)";

    db.query(query, [decimal, binary], (err, result) => {
      if (err) {
        console.error("Erro ao salvar no banco de dados:", err);
        return res.status(500).json({
          error: "Erro ao salvar no banco de dados"
        });
      }
      res.json({ decimal, binary });
    });
  });
});

app.listen(port, () => {
  console.log(`Servidor rodando em http://localhost:${port}`);
});

```


Para que não haja necessidade de instalar o MySQL e um SGBD em nossa máquina, vamos criar um *docker-compose*. O *docker-compose* abaixo cria um container MySQL configurando usuário, senha e deixando a porta 3306 exposta para que possamos utilizar esse recurso. Algumas linhas abaixo ele também cria um *container* do phpmyadmin, que é um gerenciador de banco de dados MySQL, é através dessa ferramenta que vamos criar nossa base.

Saiba Mais

Utilizar ferramentas através do *Docker* é uma boa prática, pois apenas com um *docker-compose* podemos executar em diversos sistemas operacionais como Windows, Mac, Linux e obter o mesmo resultado.

Crie um arquivo `docker-compose.yml` e insira o seguinte código:

```
services:
  mysql:
    image: mysql:8.0
    container_name: mysql-container
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: password
      MYSQL_DATABASE: conversoes_db
      MYSQL_USER: user
      MYSQL_PASSWORD: userpassword
```

```
      ports:
        - "3306:3306"
      volumes:
        - mysql_data:/var/lib/mysql

  phpmyadmin:
    image: phpmyadmin:latest
    container_name: phpmyadmin-container
    restart: always
    environment:
      PMA_HOST: mysql
      PMA_USER: root
      PMA_PASSWORD: password
    ports:
      - "8080:80"

volumes:
  mysql_data:
```

Após criar os dois arquivos, vamos agora instalar os pacotes que nosso código utiliza com o terminal aberto no diretório digite o comando:

```
npm init -y
```

Você deve receber uma saída semelhante com a da imagem abaixo:

```
→ npm init -y
Wrote to /Users/jonatascarvalho/Documents/code-sql/package.json:

{
  "name": "code-sql",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}
```

Figura 1 – Retorno após iniciar o projeto

Fonte: Acervo do conteudista

#ParaTodosVerem: a imagem reproduz o retorno que o usuário deve receber após executar o comando `npm init -y`, ele exibe um json com o nome do diretório, versão, classe principal, palavras-chaves, autor, descrições entre outras informações. Fim da descrição.

Agora vamos fazer a instalação do *framework Express*, basta digitar o seguinte comando em seu terminal:

```
npm install express
```

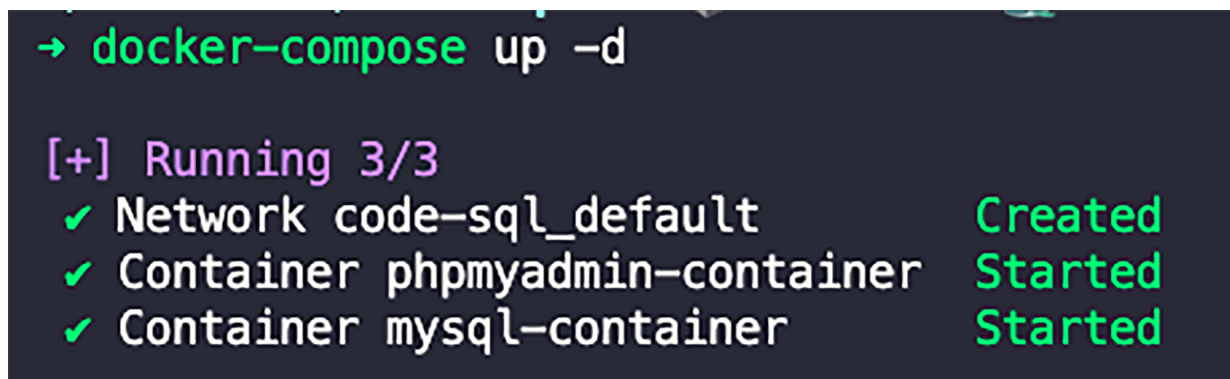
Por último, vamos instalar o pacote `mysql2` para podermos fazer a integração com o banco de dados, basta digitar o comando a seguir no terminal:

```
npm install mysql2
```

Vamos agora executar nossos *containers*, basta digitar o seguinte comando no terminal:

```
docker-compose up -d
```

Você receberá uma saída semelhante à imagem a seguir:



```
→ docker-compose up -d

[+] Running 3/3
✓ Network code-sql_default      Created
✓ Container phpmyadmin-container Started
✓ Container mysql-container     Started
```

Figura 2 – Retorno após subir os *containers*

Fonte: Acervo do conteudista

#ParaTodosVerem: imagem reproduz o retorno que o usuário deve receber após a execução do comando `docker-compose up -d`, exibe que a *network* dos *containers* foi criada, e exibe que o *container* do *phpmyadmin* e do *mysql* foram startados. Fim da descrição.

Agora, em seu navegador, insira o endereço *localhost:8080* e, então, você estará na página principal do phpMyAdmin, que é por onde vamos manipular nosso banco de dados. Nessa primeira tela, vamos clicar na aba SQL, inserir o comando e clicar no botão Go para executá-lo, como se segue na imagem abaixo:

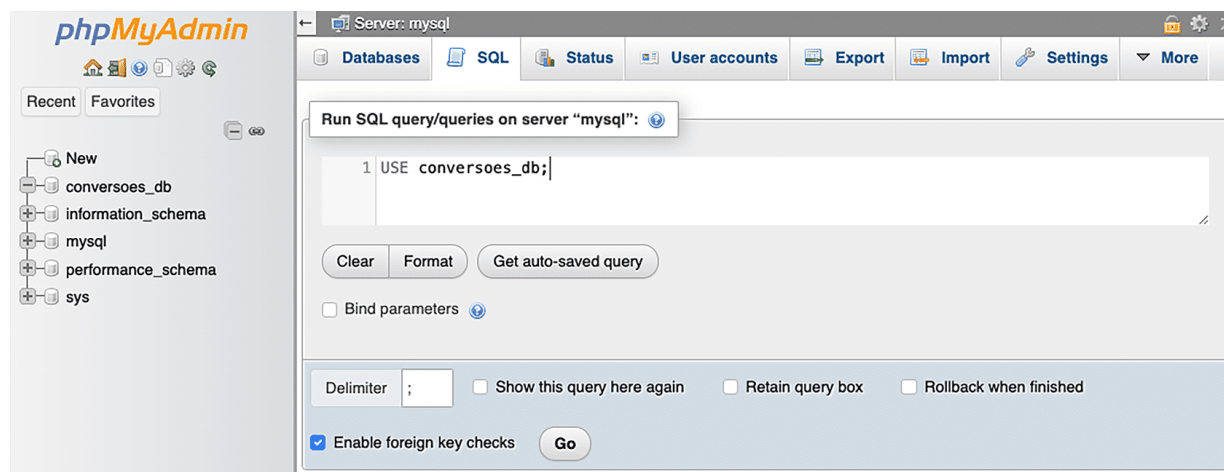


Figura 3 – Página inicial do phpMyAdmin

Fonte: Acervo do conteudista

#ParaTodosVerem: a imagem reproduz a página inicial do phpMyAdmin com o comando `USE conversoes_db;` que é utilizado para usar um banco de dados. Do lado esquerdo, temos as bases de dados que estão criadas e, na parte superior, temos algumas opções de menu, como Databases, SQL, Status, User accounts, Export, Import e Settings. Fim da descrição.

O comando anteriormente utilizado seleciona o banco de dados **conversoes_db** para o utilizarmos. No menu à esquerda, no nome do banco, novamente clique em SQL, insira o comando como na imagem abaixo e, então, novamente clique no botão **Go**.

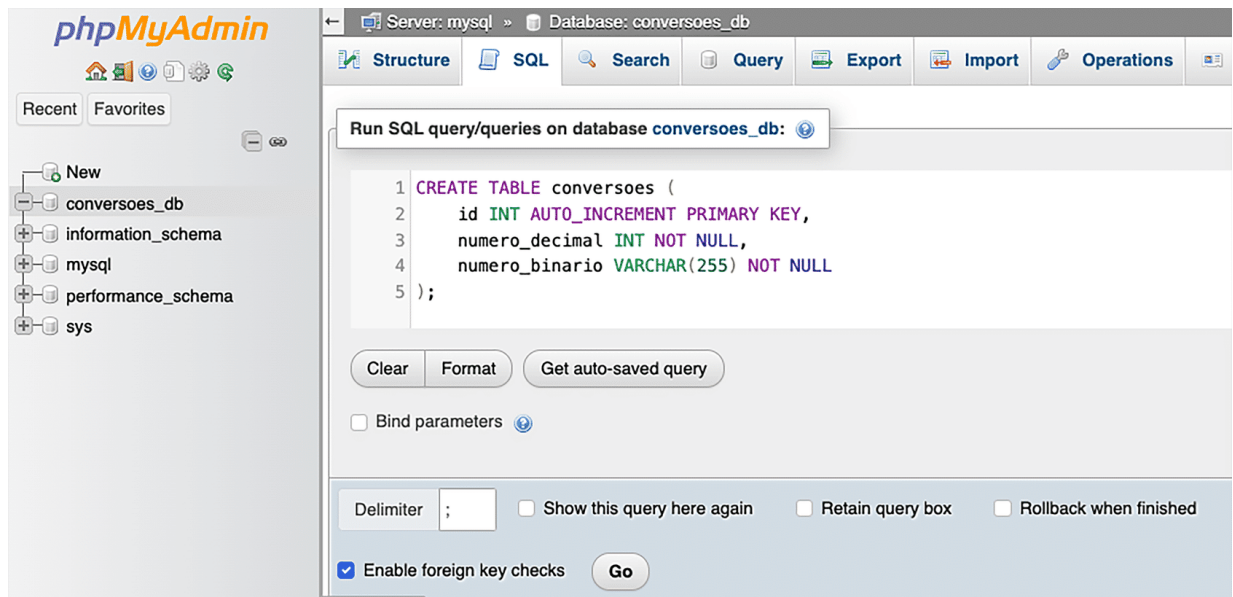


Figura 4 – Console de inserção de comandos SQL

Fonte: Acervo do conteudista

#ParaTodosVerem: a imagem reproduz o console SQL do banco de dados `conversoes_db` com o código para criar a tabela `conversoes`. Do lado esquerdo, temos as bases de dados que estão criadas e, na parte superior, temos algumas opções de menu como *Structure*, *SQL*, *Search*, *Query*, *Export*, *Import* e *Operation*. Fim da descrição.

Após a execução do comando, abrindo o menu à esquerda, veremos a tabela criada e suas colunas como segue na imagem abaixo:

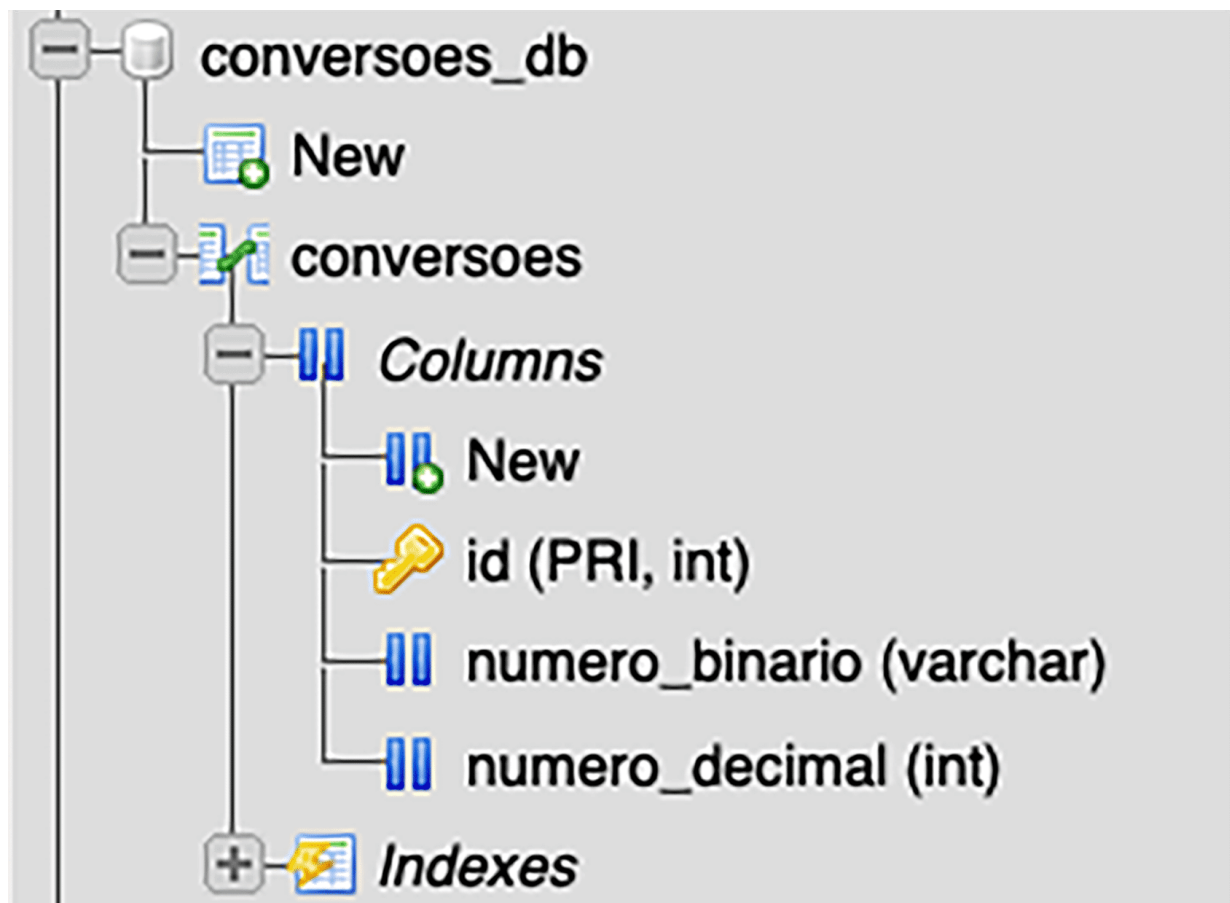


Figura 5 – Menu lateral do phpMyAdmin

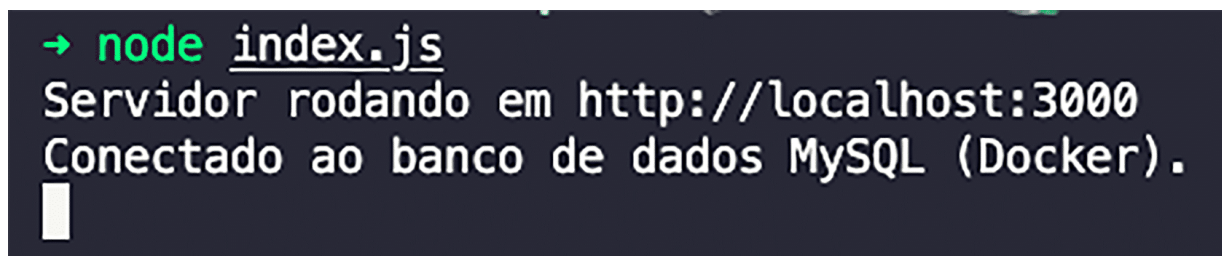
Fonte: Acervo do conteudista

#ParaTodosVerem: a imagem reproduz as colunas do banco de dados conversões. Ela exibe as colunas id, numero_binario, numero_decimal e os tipos delas, que são PRI int, varchar e int. Fim da descrição.

Agora que estamos com os *containers* rodando e nosso banco de dados em pé, vamos executar nossa aplicação. Basta executar o seguinte comando no terminal:

```
node index.js
```

O terminal deve reproduzir a seguinte saída após a execução do comando acima:



```
→ node index.js  
Servidor rodando em http://localhost:3000  
Conectado ao banco de dados MySQL (Docker).  
█
```

Figura 6 – Saída informando que a aplicação está rodando na porta 3000

Fonte: Acervo do conteudista

#ParaTodosVerem: a imagem reproduz a saída da aplicação, que informa a conexão com o banco de dados MySQL e pode ser acessado na URL localhost:3000. Fim da descrição.

Para demonstração, foram feitas 3 requisições no endereço `http://localhost:3000/to-binary/`, com os valores: 55, 7 e 82. Feito isso, basta inserir o comando **SELECT * FROM conversoes** no console do SQL e clicar no botão **Go** e, então, você verá todos os dados inseridos na base, como na imagem abaixo:

✓ Showing rows 0 - 2 (3 total, Query took 0.0008 seconds.)

```
SELECT * FROM conversoes;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Sort by key: None

Extra options

		id	numero_decimal	numero_binario
☐	Edit	Copy	Delete	1 7 111
☐	Edit	Copy	Delete	2 55 110111
☐	Edit	Copy	Delete	3 82 1010010

Figura 7 – Tabela conversões com dados inseridos

Fonte: Acervo do conteudista

#ParaTodosVerem: a imagem reproduz o console do phpMyAdmin, que exibe o resultado da consulta SQL no banco conversoes após o comando executado: `SELECT * FROM conversoes`. Abaixo, é exibido o resultado da Query, com as colunas `id`, `numero_decimal` e `numero_binario`. As linhas que vêm da consulta são: Primeira linha: `'id': 1 'numero_decimal': 7 'numero_binario': 111`. Segunda linha: `'id': 2 'numero_decimal': 55 'numero_binario': 110111`. Terceira linha: `'id': 3 'numero_decimal': 82 'numero_binario': 1010010`. Fim da descrição.

Criar integração de aplicações com banco de dados sempre será um trabalho árduo, perceba que fizemos diversos passos para colher o resultado final, porém conseguimos concluir o exemplo com sucesso.

Integração Entre uma Aplicação em Node.js e o MongoDB

Finalizando nosso conteúdo, vamos criar uma aplicação utilizando Node.js integrando com o banco de dados MongoDB. Assim como fizemos em toda a nossa disciplina, novamente não vamos instalar os aplicativos em nossa máquina, usaremos o

MongoDB via *container* do *Docker*. Em nosso `docker-compose.yml`, vamos colocar as linhas de código abaixo:

```
version: "3.8"
services:
  mongo:
    image: mongo:5.0
    container_name: mongo
    environment:
      - MONGO_INITDB_ROOT_USERNAME=root
      - MONGO_INITDB_ROOT_PASSWORD=password
    restart: unless-stopped
    ports:
      - "27017:27017"
    volumes:
      - ./database/db:/data/db
      - ./database/dev.archive:/Databases/dev.archive
      - ./database/production:/Databases/production

  mongo-express:
    image: mongo-express
    container_name: mexpress
    environment:
      - ME_CONFIG_MONGODB_ADMINUSERNAME=root
      - ME_CONFIG_MONGODB_ADMINPASSWORD=password
      - ME_CONFIG_MONGODB_URL=mongodb://
/root:password@mongo:27017/?authSource=admin
      - ME_CONFIG_BASICAUTH_USERNAME=mexpress
      - ME_CONFIG_BASICAUTH_PASSWORD=mexpress
    links:
      - mongo
    restart: unless-stopped
    ports:
      - "8081:8081"
```

Em nosso *container*, vamos dockerizar duas ferramentas, o MongoDB que vai ser o banco de dados NoSQL e, também, será utilizado o Mongo Express que é uma interface

gráfica para o MongoDB, já que ele quando está sendo executado em *container* não tem interface gráfica, e é utilizado apenas através de logs no terminal. Após criar o arquivo, basta digitar o comando *docker-compose up -d* para que o *docker* execute os *containers*.

Após ter criado os *containers*, vamos criar nossa aplicação que você encontrará no código logo abaixo, a aplicação é a mesma que implementamos na integração com o banco de dados relacional, porém foram alteradas algumas partes do código para que a aplicação pudesse se conectar com o banco de dados MongoDB.

```
const express = require("express");
const { MongoClient } = require("mongodb");
const app = express();
const port = 3000;
// Configuração da conexão com o MongoDB
const mongoUrl = "mongodb://root:password@localhost:27017/conversoes_db?authSource=admin"; // URL do MongoDB no container Docker
const dbName = "conversoes_db"; // Nome do banco de dados
let db;
// Conectar ao banco de dados
MongoClient.connect(mongoUrl, { useUnifiedTopology: true })
  .then((client) => {
    console.log("Conectado ao MongoDB.");
    db = client.db(dbName); // Seleciona o banco de dados
  })
  .catch((err) => {
    console.error("Erro ao conectar ao MongoDB:", err);
    process.exit(1); // Encerra a aplicação
  });
// Endpoint para converter decimal para binário e salvar no banco de dados
app.get("/to-binary/:decimal", async (req, res) => {
  const decimal = parseInt(req.params.decimal, 10);
  if (isNaN(decimal)) {
    return res.status(400).json
```

```
{ { error: "Número decimal inválido" } } );
    }
const binary = decimal.toString(2);
try {
    // Insere a conversão no banco de dados
    const result = await db.collection("conversoes").insertOne(
        numero_decimal: decimal,
        numero_binario: binary,
    );
    res.json({ decimal, binary, insertedId: result.insertedId });
} catch (err) {
    console.error("Erro ao salvar no banco de dados:", err);
    res.status(500).json(
        { { error: "Erro ao salvar no banco de dados" } } );
    }
});
app.listen(port, () => {
    console.log(`Servidor rodando em http://localhost:${port}`);
});
```

Com o código construído, falta agora instalarmos os complementos que nosso código usa. Para isso, basta executar os seguintes comandos:

```
npm install express
npm install mongodb
```

Agora basta executar a aplicação com o comando `node index.js`, e você terá uma saída similar à da imagem abaixo:

```
→ node index.js
(node:71688) [MONGODB DRIVER] Warning: useUnifiedTopology is a deprecated option: useUnifiedTopology has no effect since Node.js Driver version 4.0.0 and will be removed in the next major version
(Use `node --trace-warnings ...` to show where the warning was created)
Servidor rodando em http://localhost:3000
Conectado ao MongoDB.
```

Figura 8 – Saída após a inicialização da apli

Fonte: Acervo do conteudista

#ParaTodosVerem: a imagem reproduz a saída que a aplicação gera após a inicialização, digitando o comando `node index.js`. Após a execução do comando, ele exibe a mensagem: Servidor rodando em `http://localhost:3000`. Conectado ao MongoDB. Fim da descrição.

Para demonstração, foram realizadas 3 requisições na URL

`http://localhost:3000/to-binary/` com os valores 10, 37, 1517.

Após a realização das requisições em nossa aplicação, vamos acessar a URL: `http://localhost:8081` em nosso navegador e você encontrará a página inicial do Mongo Express já com nosso banco de dados `conversoes_db` na lista de bancos como segue na imagem abaixo:

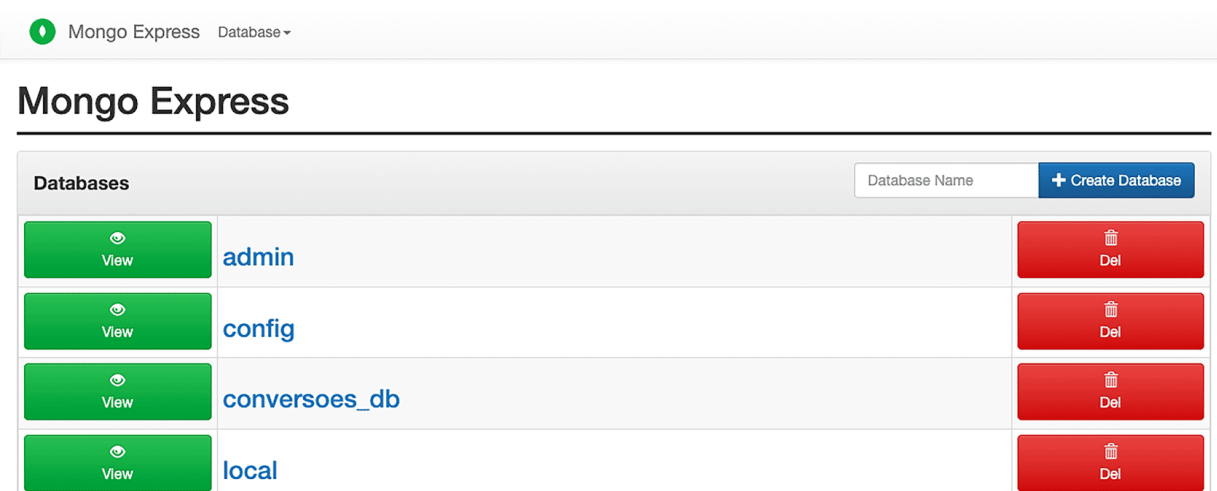


Figura 9 – Página inicial do Mongo Express

Fonte: Acervo do conteudista

#ParaTodosVerem: a imagem reproduz a página inicial do sistema gerenciador Mongo Express. Na parte superior, temos uma barra de pesquisa e, abaixo, temos todos os bancos de dados listados. Do lado esquerdo, temos o botão *View* para exibir os dados dos bancos e, do lado direito, temos o botão *Delete* para deletar os bancos de dados. Fim da descrição.

Clicando no botão verde *View* ao lado do nome `conversoes_db`, você será redirecionado na página da nossa base de dados. Feito isso, basta clicar novamente no botão *View*, e você verá os dados que foram inseridos dentro do banco de dados. Como segue nas imagens abaixo:

Viewing Database: conversoes_db

Database Stats	
Collections (incl. system.namespaces)	1
Data Size	67.0 Bytes
Storage Size	20.5 KB
Avg Obj Size #	67.0 Bytes
Objects #	1
Indexes #	1
Index Size	20.5 KB

Figura 10 – Página do banco de dados conversoes_db

Fonte: Acervo do conteudista

#ParaTodosVerem: a imagem reproduz a interface gráfica do Mongo Express. O banco de dados que está sendo visualizado é o 'conversoes_db'. Na parte superior, existe uma seção chamada Collections, que faz a listagem das coleções existentes no banco de dados. No meio da figura, temos 5 botões: *View*, para visualizar a coleção; *Export*, para exportar um banco de dados; *JSON*, para exportar um arquivo do tipo .json; *Import*, para importar dados; e *Delete*, para

deletar uma coleção. Ao fim da página, temos as seguintes estatísticas sobre o banco de dados: 'Collections (incl. system.namespaces)': 1. 'Data Size': 67.0 Bytes. 'Storage Size': 20.5 KB. 'Avg Obj Size #': 67.0 Bytes. 'Objects #': 1. 'Indexes #': 1. 'Index Size': 20.5 KB.'. Fim da descrição.

Viewing Collection: conversoes

The screenshot shows the Mongo Express web interface for the 'conversoes' collection. At the top, there are two green buttons: 'New Document' and 'New Index'. Below these is a blue bar with a 'Simple' search mode selected and an 'Advanced' option. A search bar contains 'Key' and 'Value' fields, a 'String' dropdown, and a 'Find' button. A red banner below the search bar says 'Delete all 3 documents retrieved'. The main area displays a table with three columns: '_id', 'numero_decimal', and 'numero_binario'. There are three rows of data, each with a link icon and a delete icon next to the '_id' field.

_id	numero_decimal	numero_binario
678798c885d6b7263c286407	10	1010
67879a0385d6b7263c286408	37	100101
67879a0a85d6b7263c286409	1517	1011101101

Figura 11 – Página que exhibe os dados inseridos no banco de dados

Fonte: Acervo do conteudista

#ParaTodosVerem: a imagem exhibe a interface gráfica do Mongo Express da coleção chamada conversoes. Na parte superior, existem dois botões para a criação de novos documentos ou novos índices. Mais abaixo, existe uma barra de busca com opção de busca simples ou avançada. Na sequência, é exibido os dados que foram inseridos na base, que são: Primeira linha: id': 678798c885d6b7263c286407, numero_decimal: 10, numero_binario: 1010. Segunda linha: '_id': 67879a0385d6b7263c286408, numero_decimal: 37, numero_binario: 100101. Terceira linha: '_id': 67879a0a85d6b7263c286409, numero_decimal: 1517, numero_binario: 1011101101. Fim da descrição.

Por fim, terminamos a integração da nossa aplicação com o banco de dados MongoDB.



Referências

BRADSHAW, S.; BRAZIL, E. **MongoDB: the definitive guide – powerful and scalable data storage**. 3. ed. Sebastopol: O'Reilly Media, 2019.

NIELD, T. **Introdução à linguagem SQL: abordagem prática para iniciantes**. Sebastopol: O'Reilly Media, 2016.